TASKS & FULL-STACK PROJECT REQUIREMENTS

Authentication • Authorization • JWT • Protected Routes • React–Express Integration

Part A – Six Hands-on Practice Tasks

Task 1 – User Registration and Password Hashing

Create a registration form with name, email, and password fields. Send the details to an Express API and store the user in MongoDB. Hash the password before saving it. Do not store plain-text passwords. Show a success message after successful registration and a clear message if the email already exists.

Evidence to show: working screen or API test, the relevant code file, and a short explanation of how the feature works.

Task 2 – Login and JWT Authentication

Create a login form using email and password. Verify the credentials on the backend and return a JWT after successful login. Display a suitable error message for incorrect credentials. Explain what the token is used for and do not expose the user's password in the response.

Evidence to show: working screen or API test, the relevant code file, and a short explanation of how the feature works.

Task 3 – Protected API Route

Create a protected endpoint such as GET /api/profile. It must return profile information only when a valid token is supplied. Test the endpoint without a token, with an invalid token, and with a valid token. The expected responses are 401 Unauthorized for missing or invalid authentication and 200 OK for a valid authenticated request.

Evidence to show: working screen or API test, the relevant code file, and a short explanation of how the feature works.

© 2026 TRAINER – HARI SRINI | LEARN • BUILD • AUTOMATE

Task 4 – Role-Based Authorization

Support at least two roles: student and admin. Allow an authenticated student to view permitted information, while an admin can access an admin-only endpoint. Verify permissions on the backend, not only by hiding buttons in React. Return 403 Forbidden when a logged-in user lacks permission.

Evidence to show: working screen or API test, the relevant code file, and a short explanation of how the feature works.

Task 5 – React and Express Integration

Build a React page that calls an Express API and displays the returned MongoDB data. Include loading, success, empty-data, and error states. Keep the backend URL in an environment variable and configure CORS for the local frontend origin.

Evidence to show: working screen or API test, the relevant code file, and a short explanation of how the feature works.

Task 6 – Search, Filter, and AI-Assisted Review

Add a search box that filters records by a relevant field, such as internship title or room name. Test matching and non-matching searches. Ask an AI coding assistant to review one function for readability, validation, security, and edge cases. Record at least two suggestions you accepted or rejected and explain why.

Evidence to show: working screen or API test, the relevant code file, and a short explanation of how the feature works.

© 2026 TRAINER – HARI SRINI | LEARN • BUILD • AUTOMATE

Common Technical Requirements for All Three Projects

- Frontend: React with Vite; use clean components and forms with labels.
- Backend: Node.js and Express with a clear routes/controllers/models structure.
- Database: MongoDB Atlas or a locally available MongoDB instance, accessed through Mongoose.
- Configuration: use .env for secrets and connection strings; provide .env.example without real credentials.
- API behaviour: validate incoming data and return suitable HTTP status codes and readable error messages.
- Security: never store plain-text passwords; protect private routes; enforce permissions on the backend; never commit secrets.
- Testing: test important APIs in Postman and verify the user flows in the browser.
- Documentation: include README, setup instructions, environment variables, API summary, and test accounts only if they are safe demo credentials.
- Local execution: document the commands and ports for starting the backend and frontend. Confirm both are running on localhost and communicate correctly.
- AI usage: AI-generated code must be reviewed, tested, and understood. Students must be ready to explain their own code.

Part B – Project 1: Personal Portfolio Website (Full Stack)

Objective: Build a personal portfolio that presents your profile and skills and retrieves at least some content from a backend API. The portfolio must run locally and use a React frontend connected to an Express backend.

Required Features

- Home section with your full name, profile photo or placeholder, short introduction, and career objective.
- About section describing your education, interests, and professional goals.
- Skills section showing technical skills relevant to your learning path.
- Projects section with at least three projects. For each project, include its title, short description, technologies used, and GitHub link if available.
- Education section with course/program, institution, and year or expected completion.
- Contact section with email or a contact form. Do not publish private personal details that you do not want publicly visible.
- Responsive layout that works on desktop and mobile screen sizes.
- Backend API that returns portfolio information such as profile, skills, education, or projects.
- Store portfolio content in MongoDB and retrieve it through Express APIs; do not hardcode all displayed project data in the React component.
- Include clear navigation, readable text, consistent styling, and working links.

Minimum API Expectations

- GET /api/portfolio – retrieve the main portfolio content.
- GET /api/projects – retrieve the project list.
- Optional: POST /api/contact – validate and record a contact message in MongoDB.

Acceptance Checklist

- The React frontend loads on localhost and successfully fetches backend data.
- The Express backend connects to MongoDB and returns valid JSON.

- Project information is displayed from the API.
- Navigation, links, and responsive layout work without major errors.
- Provide a README with setup steps and the required environment variables.

Part C – Project 2: Internship Portal (Full Stack)

Objective: Build an internship portal where users can register, log in, view internship opportunities, search/filter listings, and access features according to their roles.

Required Features

- Registration and login using email and password.
- Password hashing on the backend and JWT-based authentication.
- Public registration must assign the student role by default. Users must not be able to make themselves administrators by changing frontend form data.
- Student dashboard with a list of internship opportunities.
- Internship listing fields: title, company, location or remote status, eligibility/skills, application deadline, and application link or instructions.
- Search internships by title, company, or skill; add at least one filter such as location, remote status, or eligibility.
- Role-based access enforced by backend middleware. Students can view listings; admin users can create, update, or delete internship listings.
- Protected profile or dashboard route that requires a valid token.
- Clear messages for invalid login, empty search results, failed API requests, and successful operations.
- Store users and internship records in MongoDB using Mongoose models.
- Connect React to Express APIs; do not use only static arrays as the final data source.

Minimum API Expectations

- POST /api/auth/register – register a student account.
- POST /api/auth/login – authenticate a user.
- GET /api/auth/me – return the current authenticated user's details.
- GET /api/internships – retrieve internship listings and support search/filter parameters.
- POST /api/internships – admin-only create operation.
- PUT /api/internships/:id – admin-only update operation.
- DELETE /api/internships/:id – admin-only delete operation.

Acceptance Checklist

- A student can register, log in, view internships, search, and filter.
- An unauthenticated user cannot access protected endpoints.
- A student cannot perform admin-only operations, even by calling the API directly.
- An admin can create, update, and delete internship listings.
- Passwords are hashed; secrets and database credentials are stored in .env, not committed to source control.
- Test the API using Postman and the UI using the browser.

Part D – Project 3: Room Availability and Allocation System (Full Stack)

Objective: Build a room availability system for a college or training centre. Users can check available rooms for a selected date, time, capacity, and purpose. An authorized user can allocate a room while preventing conflicting bookings.

Required Features

- Room list with room name/number, building or location, capacity, facilities, and availability status.
- Search rooms by name/number and filter by minimum capacity or facilities such as projector, Wi-Fi, or air conditioning.
- Availability form with date, start time, end time, required capacity, and purpose of booking.
- Show rooms that meet the requested capacity and facilities and are available for the selected time period.
- Create a room booking with the requester name or authenticated user, room, date, start time, end time, and purpose.
- Validate that the end time is later than the start time and that required fields are present.
- Prevent overlapping bookings for the same room. A booking must be rejected if its time conflicts with an existing confirmed booking.
- Show bookings and their status, such as pending, confirmed, or cancelled.
- Allow an authorized admin/coordinator to add or edit rooms and confirm or cancel bookings.
- Store rooms and bookings in MongoDB; connect React to Express APIs.
- Display clear messages when no room is available, a booking conflicts, or a request succeeds.

Minimum API Expectations

- GET /api/rooms – list rooms; support search/filter parameters where appropriate.
- GET /api/rooms/availability?date=YYYY-MM-DD&startTime=HH:mm&endTime=HH:mm&capacity=30 – return matching available rooms.
- POST /api/bookings – create a booking after validating availability.
- GET /api/bookings – list bookings for the authorized view.
- PATCH /api/bookings/:id/status – authorized coordinator/admin updates booking status.
- POST /api/rooms and PUT /api/rooms/:id – admin/coordinator room management.

Acceptance Checklist

- Users can search rooms and check availability for a selected date/time.
- Rooms that do not meet the capacity requirement are excluded.
- Two overlapping confirmed bookings for the same room cannot be created.
- A booking outside an existing booking's time period is allowed when other requirements are met.
- Only authorized users can manage rooms or change booking status.
- Room and booking data persists in MongoDB after restarting the backend.

© 2026 TRAINER – HARI SRINI | LEARN • BUILD • AUTOMATE

Submission Checklist

- Six hands-on tasks completed and demonstrated.
- Project 1: Personal Portfolio – frontend, backend, and database connected.
- Project 2: Internship Portal – login, authentication, authorization, CRUD, search/filter, and database connected.
- Project 3: Room Availability and Allocation – search, availability validation, booking conflict prevention, and database connected.
- All three projects run locally without relying on hardcoded mock data as the final data source.
- Each project has its own README and .env.example.
- No real secrets or passwords are included in the submitted project files.
- Student can explain the flow: **React UI → Express API → validation / authentication / authorization → MongoDB → response to React.**

****